

EC-350 AI and Decision Support Systems

Week 7 Adversarial Search

Dr. Arslan Shaukat



Acknowledgement: Lecture slides material from
Stuart Russell

1

MAEs and Games

- Till now we have only studied the searches where only one agent is exploring the problem space to find a goal
- In many applications there might be more than one agents looking for goal
- **Multi-agent environment (MAE):** every agent needs to consider the actions of the other agents, in order to optimize its own welfare
 - *Cooperative: Agents act collectively to achieve a common goal*
 - *Competitive:*
 - Agents compete against each other
 - Their goals are in conflict
 - Gives rise to the concept of **adversarial** search problems – often known as **games**.

2

Game Theory

- Views any MAE as a game provided that the impact of each agent on the other is “significant”, i.e., able to affect the actions of the other agent(s)
- In AI, “game” is a specialized concept:
- Usually deterministic, turn-taking, two player, zero-sum games are played
 - *Deterministic, fully-observable environments*
 - *Two agents whose actions must alternate*
 - *Utility values at the end of the game are always equal and opposite*
 - +1 = Chess winner
 - -1 = Chess loser.

12/11/2022

EC-350 AI and DSS

EME (NUST)

3

3

AI Games

- Chess had been tackled by Konrad Zuse, Claude Shannon, Norbert Wiener, Alan Turing
 - *Have seen lot of successes recently, e.g., DeepBlue*
- Game states are easy to represent:
 - *Agents restricted by limited actions*
 - *Outcomes defined by precise rules*
- **Games:** interesting but are also hard to solve:
 - *Chess: average branching factor of 35*
 - *If 50 moves by each player, search tree has 35^{100} nodes!*

12/11/2022

EC-350 AI and DSS

EME (NUST)

4

4

Games vs. Search Problems

- All games can be formally modeled as search problems
- In typical search problems, we optimize a measure to acquire the goal: there is no opponent
- Searches where two or more players with contrary goals are exploring the same solution space are called **Adversarial Searches**
- In games, there is an "Unpredictable" opponent
 - *Need to specify a move for every possible opponent reply*
 - *Strict penalty on an inefficient move*
 - *Stringent time constraints*
 - *Requires some type of a decision to move the search forward.*

12/11/2022

EC-350 AI and DSS

EME (NUST)

5

5

Game Definition

- Game is a kind of search problem with following components:
 - *Initial state: Board position and identify player to move first*
 - *Successor Function: returns a list of (move, state) pairs, each indicate a legal move and resulting state*
 - *Terminal test: Has the game ended? States where the game has ended are called terminal states*
 - *Utility function (UF): Gives numeric value for terminal states. e.g. win (+1), loose (-1) and draw (0) in tic-tac-toe*
 - Also called the objective function or payoff function

12/11/2022

EC-350 AI and DSS

EME (NUST)

6

6

Game Setup

- Imagine 2 players of tic-tac-toe: MAX and MIN
 - MAX moves first: We can generate a **game tree**
 - The terminal states are at the leaves
- MAX should play in order to maximize its utility, which will minimize the utility for MIN
 - This is called a **Zero-Sum Game**.

12/11/2022

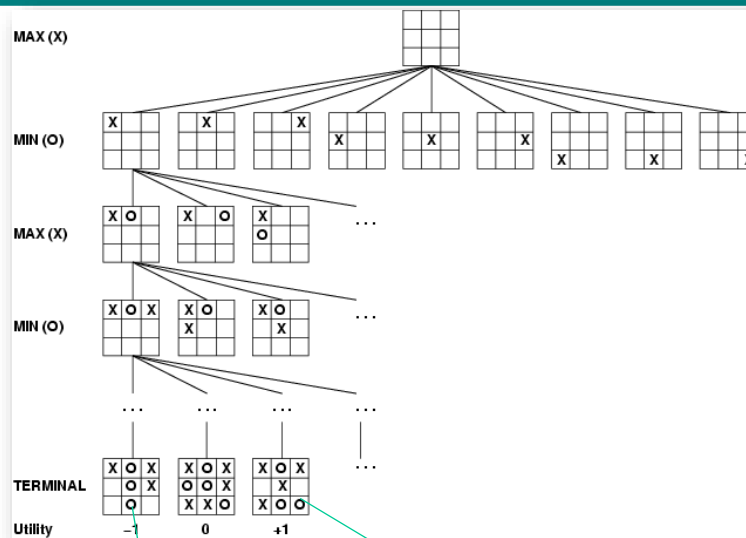
EC-350 AI and DSS

EME (NUST)

7

7

Game Tree



This terminal state is one of the worst for MAX and one of the best for MIN

This terminal state is one of the best for MAX and one of the worst for MIN

12/11/2022

EC-350 AI and DSS

EME (NUST)

8

8

Optimal Strategy For MAX

- In discovering the sequence of optimal moves by MAX, we have to consider that MIN is taking moves as well
- So, the **optimal strategy will tell how should MAX play against MIN (by considering the moves of the latter), in order to win the game**
- MAX must have a contingent strategy at start and then for each move by MIN
- An optimal strategy, in general, leads to outcomes at least as good as any other strategy when one is playing against infallible opponent

12/11/2022

EC-350 AI and DSS

EME (NUST)

9

9

Optimal Strategy

- Let us consider a shortened game tree of TIC-TAC-TOE
 - *A simple game like tic-tac-toe is too complex for us to draw the entire game tree, we shall switch to a simple game, a tree with depth of 2 (one move deep)*
 - ***Ply:** This tree consists of two half moves, a move by MAX followed by a move from MIN, each of which is called a ply.*
 - *Each of MAX and MIN has only one move*
 - ***Numbers are the utilities***

12/11/2022

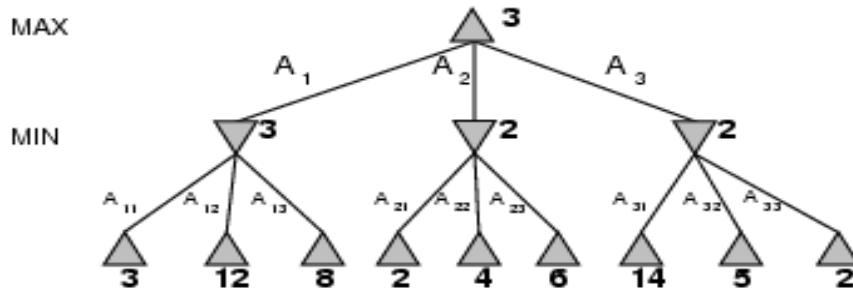
EC-350 AI and DSS

EME (NUST)

10

10

Minimax



- Assuming 3 moves by each player
- A 2-ply game tree, one move for each MAX & MIN
- $\triangle$ denotes 'MAX node' while ∇ denotes 'MIN mode'
- Leaf nodes give **utility** values for MAX

12/11/2022

EC-350 AI and DSS

EME (NUST)

11

11

Minimax

- When it is the turn of MAX, **it will always take an action in order to maximize its utility**, because it's winning configurations have high utilities
- When it is the turn of MIN, **it will always take an action in order to minimize its utility**, because it's winning configurations have low utilities
- In order to implement this, we need to define a measure in each state that **takes the move of the opponent into account**:
 - This measure is called **Minimax**.*

12/11/2022

EC-350 AI and DSS

EME (NUST)

12

12

Making an Optimal Strategy

- Optimal strategy can be determined by examining **minimax value** of each node
- The minimax value of a node is the utility (for MAX) of being in the corresponding state
- Minimax value of a terminal state is just its utility
- MAX prefers a state of maximum value while Min prefers a state of minimum value
 - $\text{Minimax-Value}(n) =$
 - Utility (n) if n is a terminal state
 - $\max_{s \in \text{Successors}(n)} \text{minimax-value}(s)$ if n is a MAX node
 - $\min_{s \in \text{Successors}(n)} \text{minimax-value}(s)$ if n is a MIN node

12/11/2022

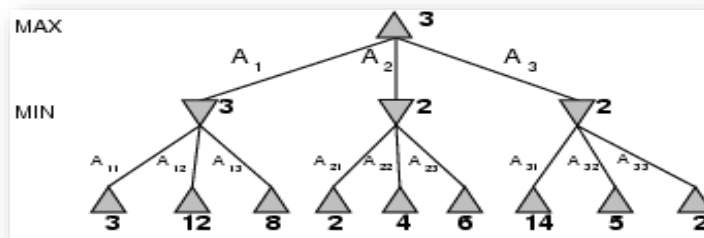
EC-350 AI and DSS

EME (NUST)

13

13

Minimax



- Consider “3” at Level 1:** MIN selects an action (A_{11}) that leads to a state of minimum utility for MAX, i.e., $\text{minimum}\{3, 12, 8\}$
- Consider “3” at Level 0:** MAX selects an action (A_1) that leads to a state of maximum utility for MIN, i.e., $\text{maximum}\{3, 2, 2\}$
 - Both are opposing what is best for the other.*

12/11/2022

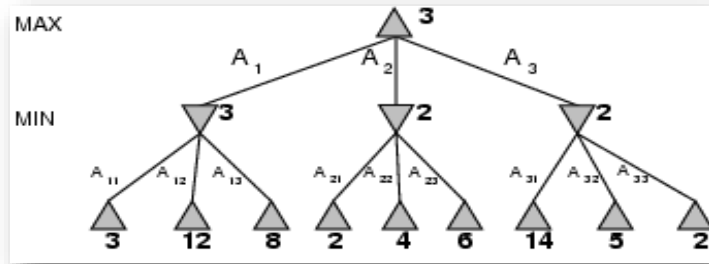
EC-350 AI and DSS

EME (NUST)

14

14

Minimax



- At each node, MAX will always select the action with highest minimax value (it wants to reach states with higher utilities)
- At each node, MIN will always select the action with lowest minimax value (it wants to reach states with lower utilities).

12/11/2022

EC-350 AI and DSS

EME (NUST)

15

15

Minimax algorithm

- Depth-First Exploration
- Recursion: Winds all the way to the terminal nodes, and then unwinds back by backing up the values
- Min(minimax) for MIN moves
- Max(minimax) for MAX moves

12/11/2022

EC-350 AI and DSS

EME (NUST)

16

16

Properties of Minimax

- Complete? Yes (if tree is finite)
- Optimal? Yes (against an optimal opponent)
- Time complexity? $O(b^m)$
- Space complexity? $O(bm)$ (depth-first exploration)
- For chess, $b \approx 35$, $m \approx 100$ for "reasonable" games $\rightarrow$ exact solution completely infeasible
 - *We need to think of a way to cut down the number of search paths.*

12/11/2022

EC-350 AI and DSS

EME (NUST)

17

17

Alpha-Beta Pruning

- When board positions are too much (such as chess) exploring each and every node is not optimal
- We do not explore that part where the value we get is not better than the value we already have

12/11/2022

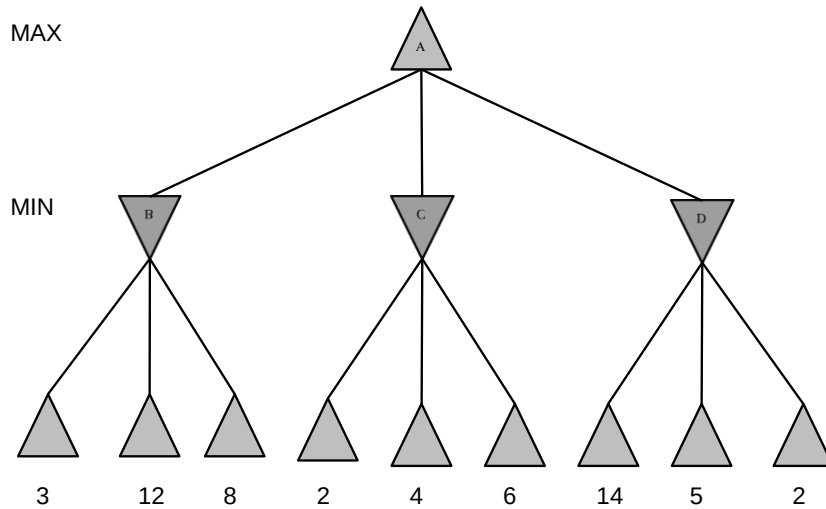
EC-350 AI and DSS

EME (NUST)

18

18

α - β Pruning – Initial Game Tree



12/11/2022

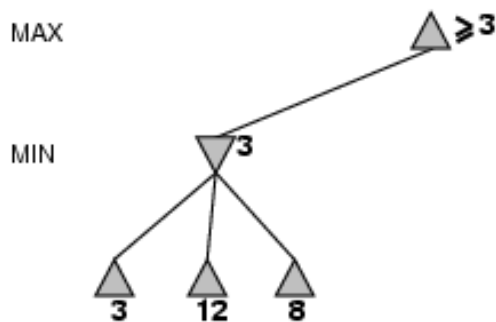
EC-350 AI and DSS

EME (NUST)

19

19

α - β pruning example



First node under B is 3 which MIN shall after exploring all three nodes. Therefore value at root (MAX) is at least 3

12/11/2022

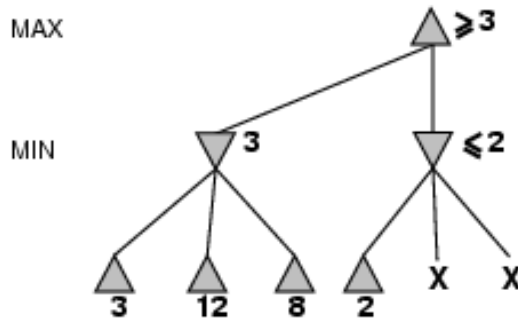
EC-350 AI and DSS

EME (NUST)

20

20

α - β pruning example



First node under C is 2 which MIN will choose. But we know that MIN has a previous value of 3 so MAX will never choose 2 so there is no point in looking at other 2 nodes so we prune them

12/11/2022

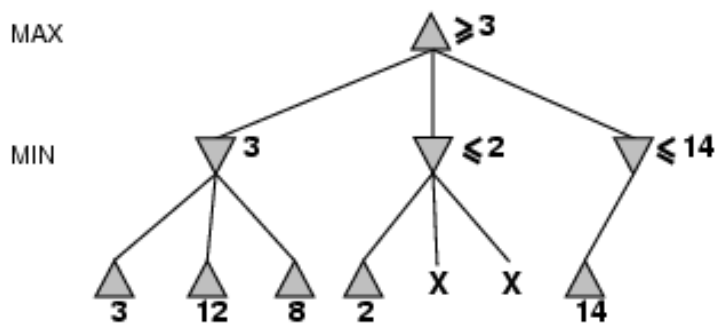
EC-350 AI and DSS

EME (NUST)

21

21

α - β pruning example



First node under D is 14 which is greater than 3 so MIN need to explore other nodes also

12/11/2022

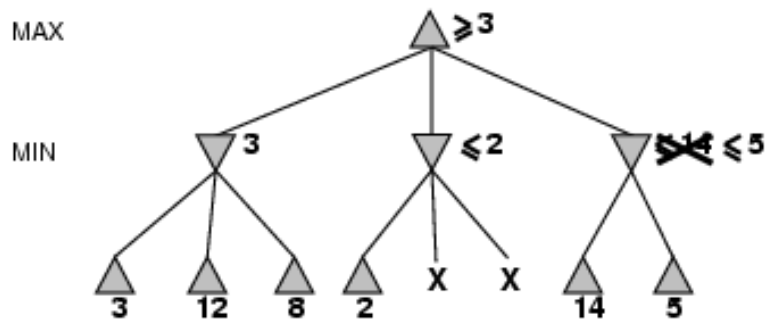
EC-350 AI and DSS

EME (NUST)

22

22

α - β pruning example



Second value under D is 5 which also is more than 3 so MIN will again look for lesser value

12/11/2022

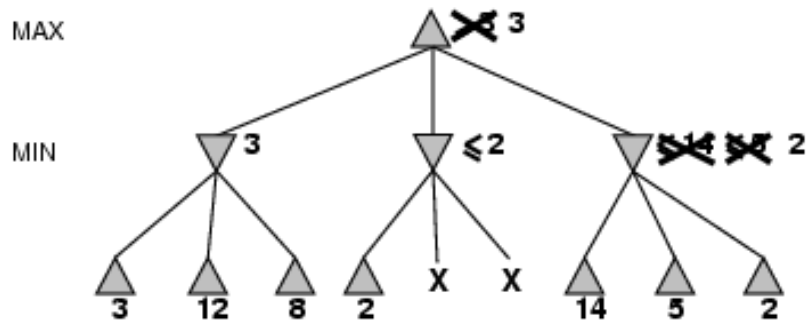
EC-350 AI and DSS

EME (NUST)

23

23

α - β pruning example



Third node is 2 which MIN will choose. So now this node is worth exactly 2. But MAX has a value of 3 so MAX will move to B

12/11/2022

EC-350 AI and DSS

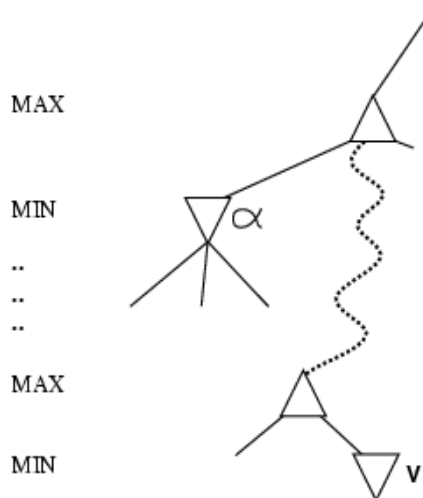
EME (NUST)

24

24

Why is it called α - β ?

- α is the value of the best (highest-value) choice found so far at any choice point along the path for *max*
- If v is worse than α , *max* will avoid it $\rightarrow$ prune that branch
- Similarly, β is value of the best (lowest value) choice found so far at any choice point along the path for *min*



12/11/2022

EC-350 AI and DSS

EME (NUST)

25

25

Properties of α - β

- Pruning **does not** affect final result
- Good move ordering improves effectiveness of pruning
- With "perfect ordering," time complexity = $O(b^{m/2})$

12/11/2022

EC-350 AI and DSS

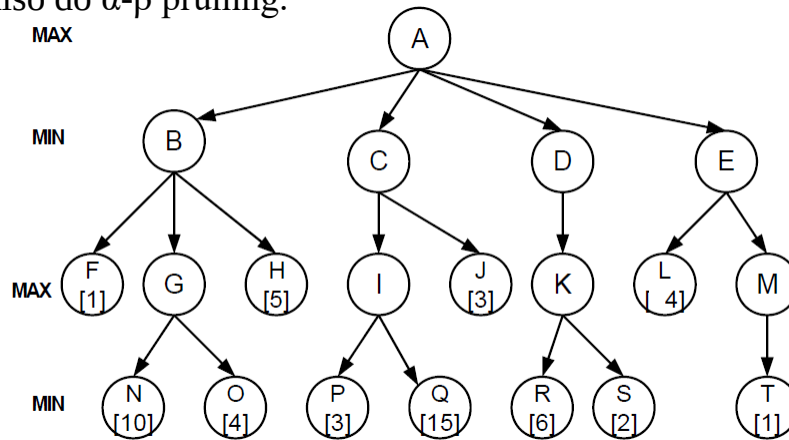
EME (NUST)

26

26

Example

- Find the path/move sequence using MIN-MAX Algorithm by finding utility values at each node.
- Also do α - β pruning.



12/11/2022

EC-350 AI and DSS

EME (NUST)

27

27